1.3.4 Variable Scope

Show Lecture Video

Variables exist in a specific **scope** based on when they are defined. This means they cannot be used outside that scope, in other parts of the code. For now, we will consider two scopes: local and global.

Local Variables

Variables defined in a function definition, including its parameters and anywhere in its function body, are called **local variables** (or just **locals**) and have **local scope**. That is, they can only be used in that function.

Here is an example that tries, and fails, to use local variables outside of a function:

```
1 def f(x):
2     return x + 5
3 print(f(4)) # prints 9
4 print(x)    # crashes!  x is not defined outside
```

Run

Here are a few more notes about local variables:

- Local variables have a **lifetime**, and exist only during a function call. Each new call to a function gets entirely new local variables.
- Even if another function has the variables with the same names, those are different variables with different scopes.

Global Variables

Variables defined outside of a function definition (that is, at the **top level**), are called **global variables** (or just **globals**) and have **global scope**. That is, they can be used everywhere.

Using global variables can lead to a variety of obscure bugs. For that reason, we generally do not allow you to use global variables in your code. However, it can be handy to use global variables in short examples, which is why we do that in the notes. Still: in general, **do not use global variables**.

Checkpoint 1

True or False: Parameters are local variables.

- ○ **True**
- ○ **False**

Submit

Checkpoint 2

True or False: Even if two functions define a local variable `x`, these are still different variables.

- ○ **True**
- ○ **False**

Submit

Continue